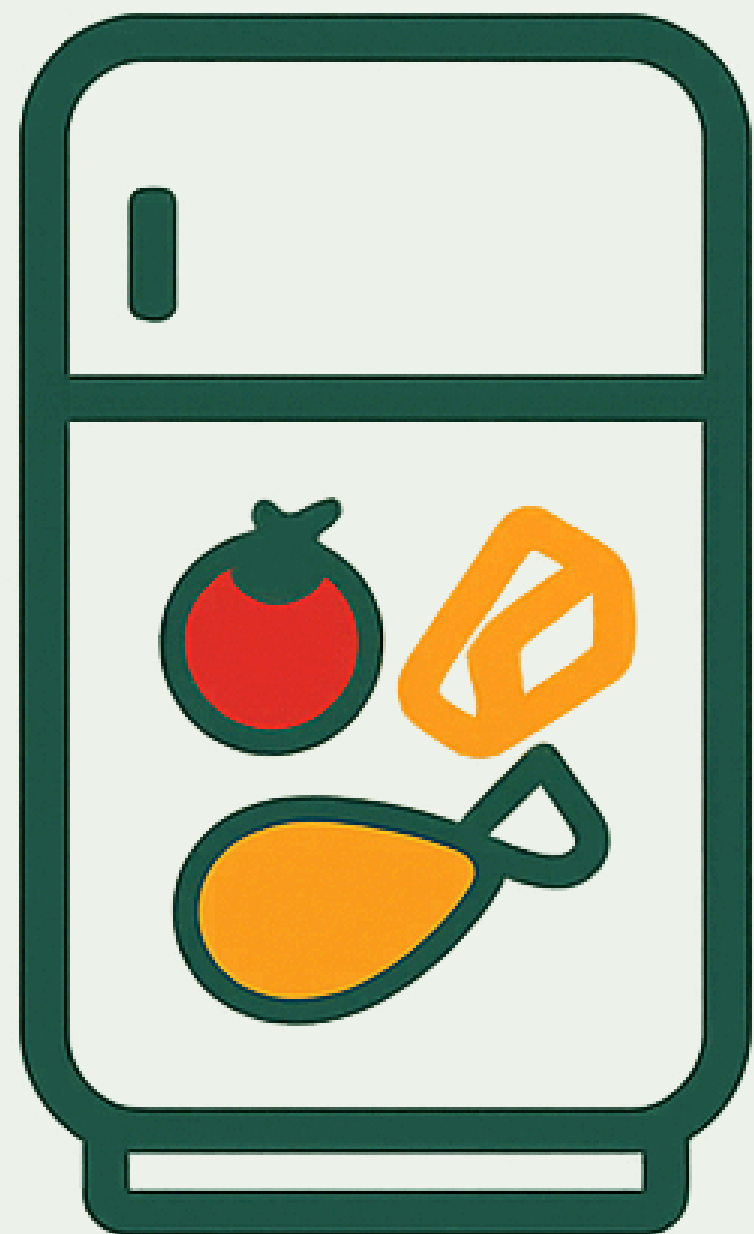




StockIT

Uma solução para gerenciar alimentos

Arthur Henrique Tristão Pinto;
Davi Rafael de Oliveira Gurgel Martins;
Gabriel Pereira Couto Rodrigues;

Contexto do **PROBLEMA**

Muitas pessoas enfrentam dificuldades para:

- Controlar alimentos vencidos;
- Evitar desperdício de alimentos;
- Manter uma rotina organizada de compras.



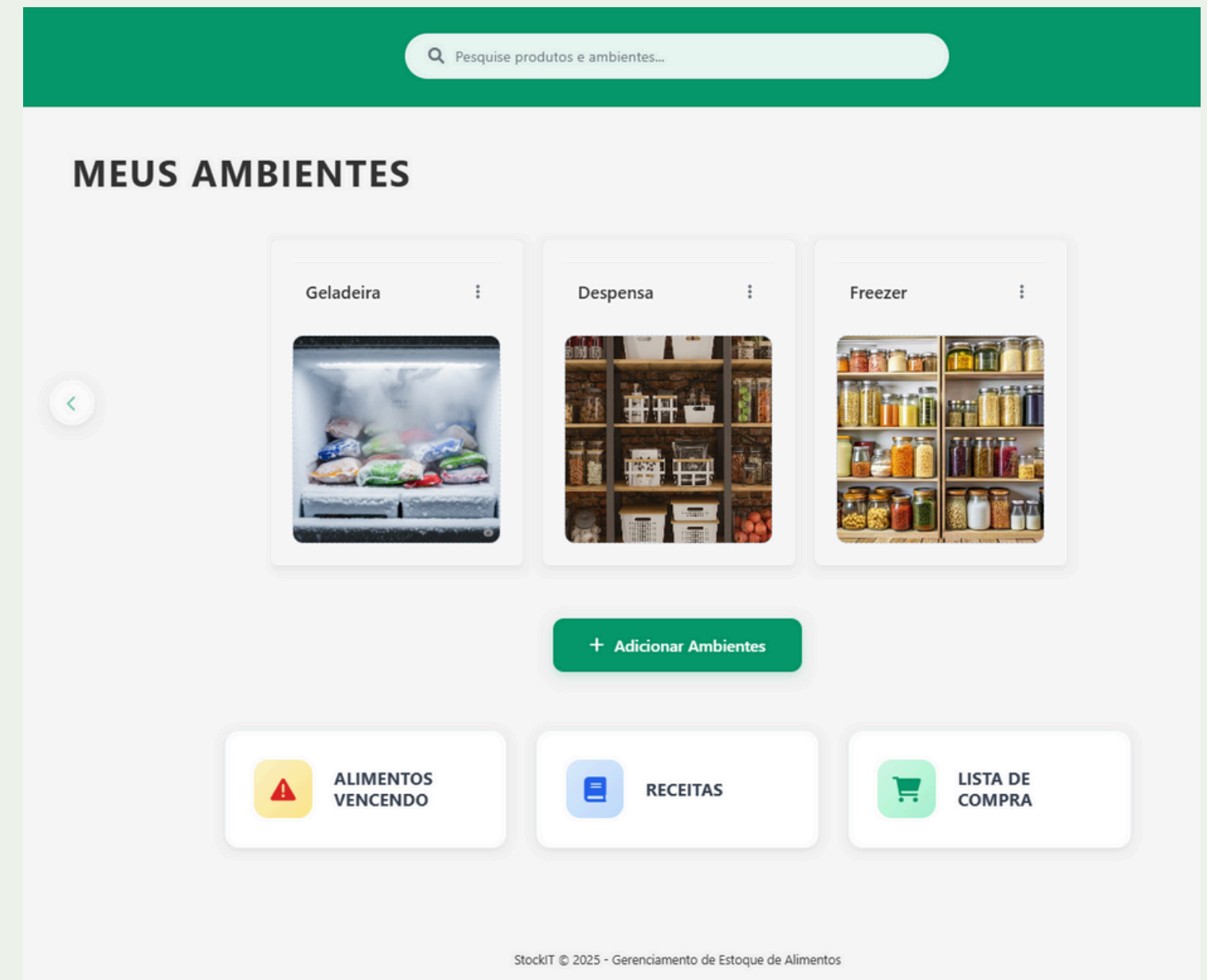
Segundo a Associação Nacional dos Restaurantes:

- Menos de 40 % das empresas de varejo investem em gestão de estoque
- Mais de 35% dos alimentos perdidos no varejo é por conta da validade

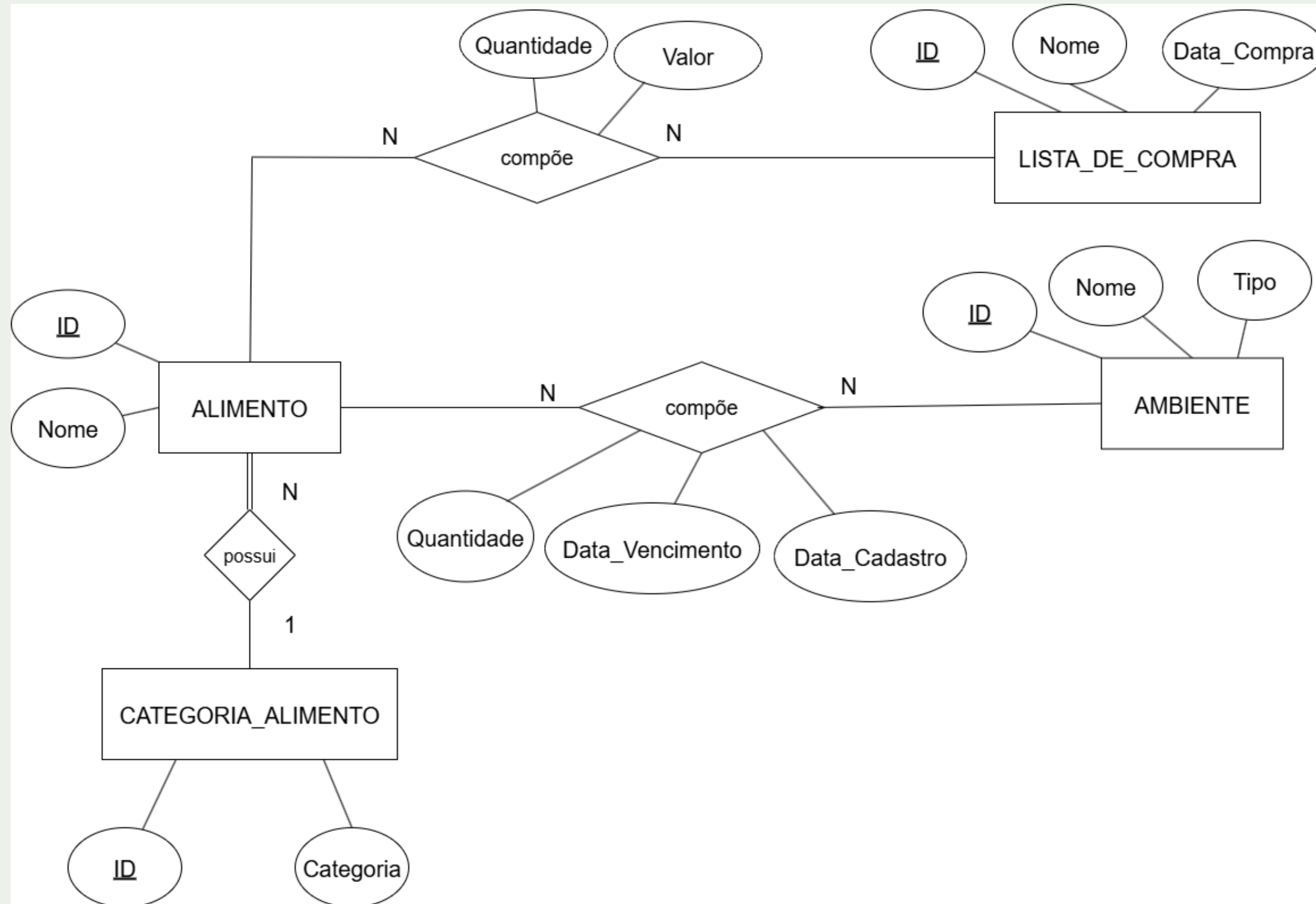
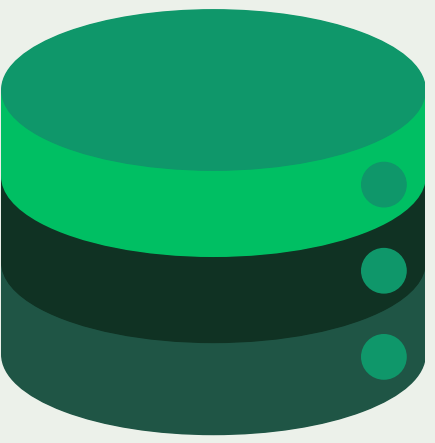
Proposta de SOLUÇÃO

Criar um sistema web que permita:

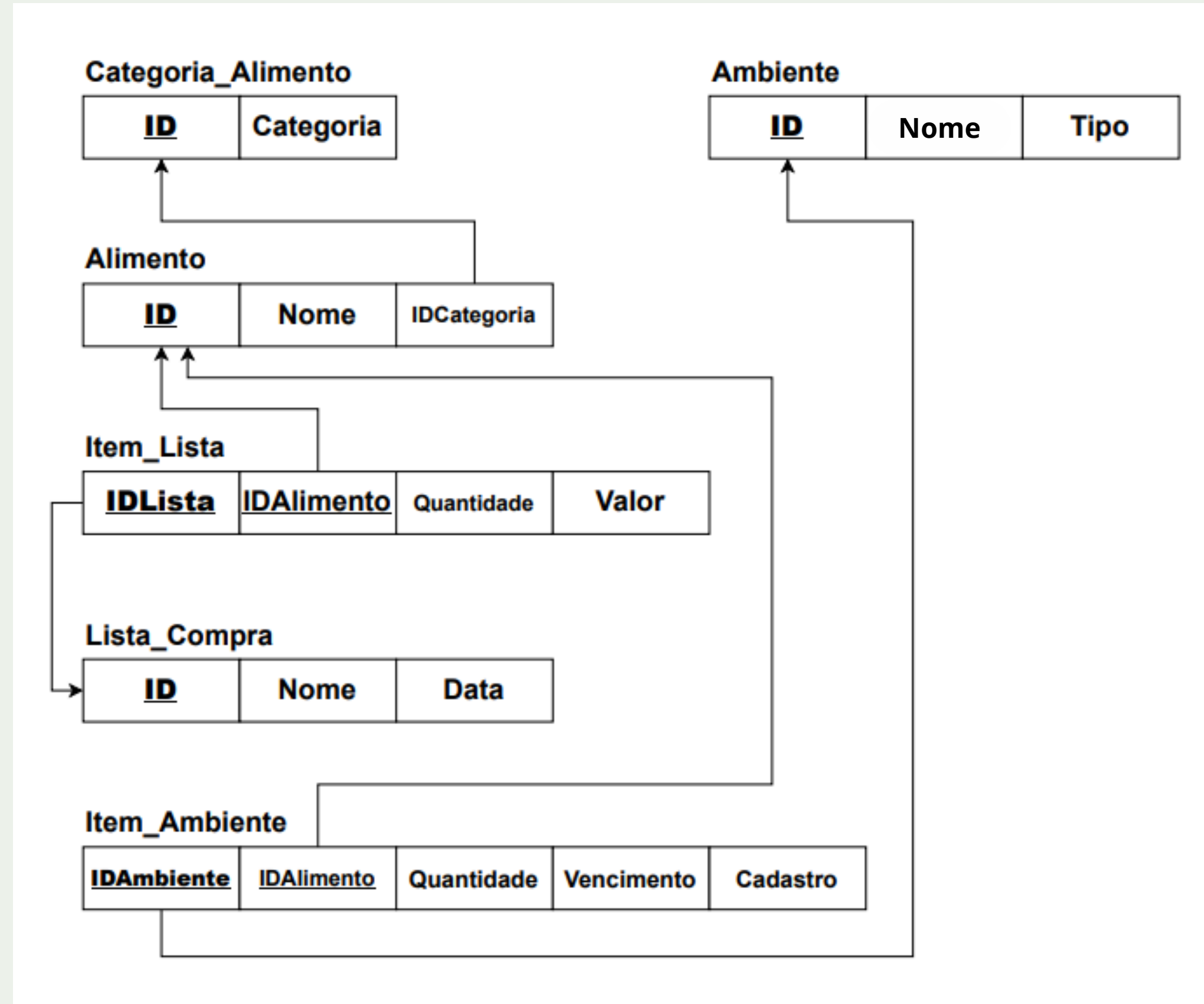
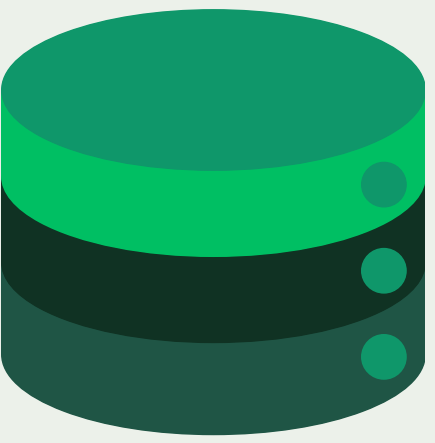
- Cadastrar ambientes e alimentos com validade
- Notificar sobre alimentos vencendo
- Organizar lista de compras
- Oferecer receitas com base em alimentos prestes a vencer



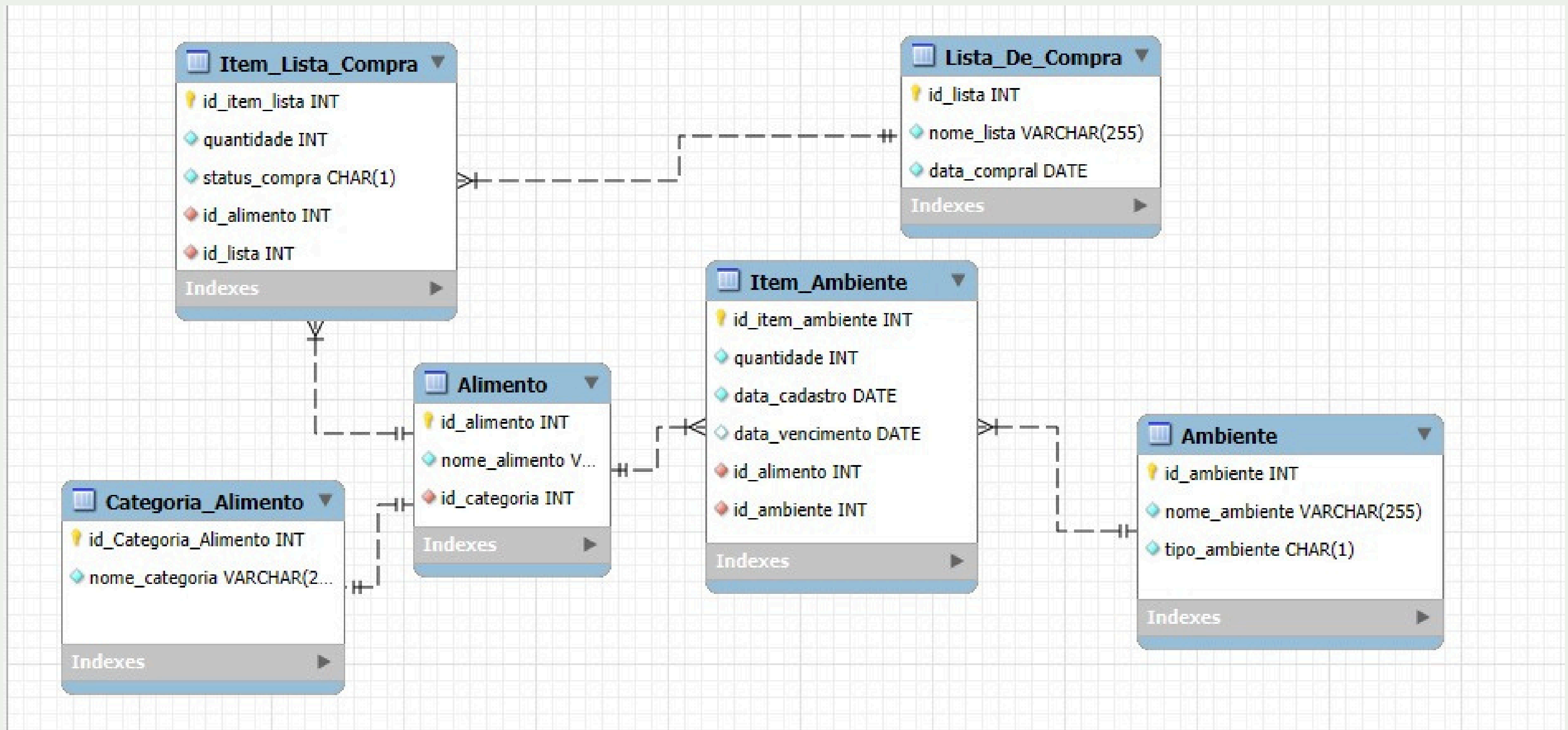
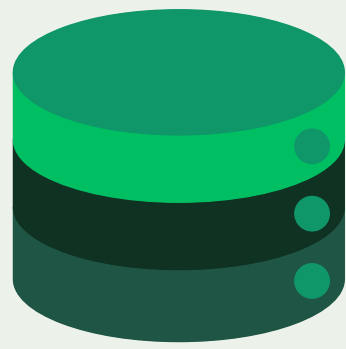
MODELO DER PETER CHEN



MODELO LÓGICO



MODELO DE IMPLEMENTAÇÃO



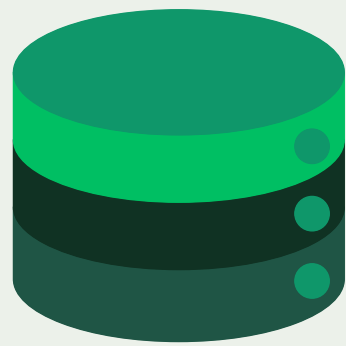
SCRIPTS SQL



```
CREATE TABLE categoria_alimento (  
    id_categoria_alimento SERIAL PRIMARY KEY,  
    nome_categoria VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE alimento (  
    id_alimento SERIAL PRIMARY KEY,  
    nome_alimento VARCHAR(255) NOT NULL,  
    id_categoria INT NOT NULL,  
    FOREIGN KEY (id_categoria)  
        REFERENCES categoria_alimento(id_categoria_alimento)  
        ON DELETE RESTRICT  
);  
  
CREATE TABLE ambiente (  
    id_ambiente SERIAL PRIMARY KEY,  
    nome_ambiente VARCHAR(255) NOT NULL,  
    tipo_ambiente CHAR(1) NOT NULL  
);  
  
CREATE TABLE item_ambiente (  
    id_item_ambiente SERIAL PRIMARY KEY,  
    quantidade INT NOT NULL,  
    data_cadastro DATE,  
    data_vencimento DATE,  
    id_alimento INT NOT NULL,  
    id_ambiente INT NOT NULL,  
    FOREIGN KEY (id_alimento)  
        REFERENCES alimento(id_alimento)  
        ON DELETE RESTRICT,  
    FOREIGN KEY (id_ambiente)  
        REFERENCES ambiente(id_ambiente)  
        ON DELETE CASCADE  
);
```

```
CREATE TABLE lista_de_compra (  
    id_lista SERIAL PRIMARY KEY,  
    nome_lista VARCHAR(255) NOT NULL,  
    data_compra DATE  
);  
  
CREATE TABLE item_lista_compra (  
    id_item_lista SERIAL PRIMARY KEY,  
    quantidade INT NOT NULL,  
    status_compra CHAR(1),  
    id_alimento INT NOT NULL,  
    id_lista INT NOT NULL,  
    FOREIGN KEY (id_alimento)  
        REFERENCES alimento(id_alimento)  
        ON DELETE RESTRICT,  
    FOREIGN KEY (id_lista)  
        REFERENCES lista_de_compra(id_lista)  
        ON DELETE CASCADE  
);
```

BANCO AZURE



Microsoft Azure

Pesquisar recursos, serviços e documentos (G+/I)

Copilot

834463@sga.pucminas.br
SGA.PUCMINAS.BR (SGAPUCMIN...)

Página inicial >

stockit Diagnosticar problemas com este recurso Gerar um modelo de Terraform para este recurso Fornecer diretrizes de solução de problemas para este recurso

Banco de Dados do Azure para servidor flexível do PostgreSQL

Pesquisar

Conectar de VS Code Excluir Redefinir a senha Restaurar Reiniciar Atualizar Parar Atualizar CLI / PS Comentários

Visão geral

Log de atividade

IAM (Controle de acesso)

Marcações

Diagnosticar e resolver problemas

Visualizador de recursos

Migração

Espelhamento do Fabric (visualizar)

Configurações

Power Platform

Segurança

Desempenho Inteligente

Monitoramento

Automação

Ajuda

A camada de computação com capacidade de intermitência é otimizada para as cargas de trabalho de desenvolvimento/teste. Para uso em produção, recomendamos as camadas de Uso Geral ou Otimizado para Memória.

Fundamentos Exibição JSON

Assinatura (mover)	: Azure for Students	Ponto de Extremidade	: stockit.postgres.database.azure.com
ID da Assinatura	: 9c0db1f1-4491-4fd9-b40c-f230e041c3d8	Ponto de extremidade vir...	: Não habilitado
Grupo de recursos (mover)	: stockit	Logon do administrador	: postgres
Nome do servidor	: stockit	Configuração	: Intermitente_B1ms_1 vCores_2 GiB RAM_32 GiB storage
Localização	: Chile Central	Versão PostgreSQL	: 16.10
Status	: Ready	Zona de disponibilidade	: 1
Criado em	: 2025-10-31 00:13:02.2114091 UTC	Alta disponibilidade	: Não habilitado
Marcações (editar)	: Adicionar marcas		

Introdução Propriedades Recomendações Monitoramento Tutoriais

Iniciar seu projeto

Conecte-se ao seu banco de dados pela primeira vez com algumas etapas simples.

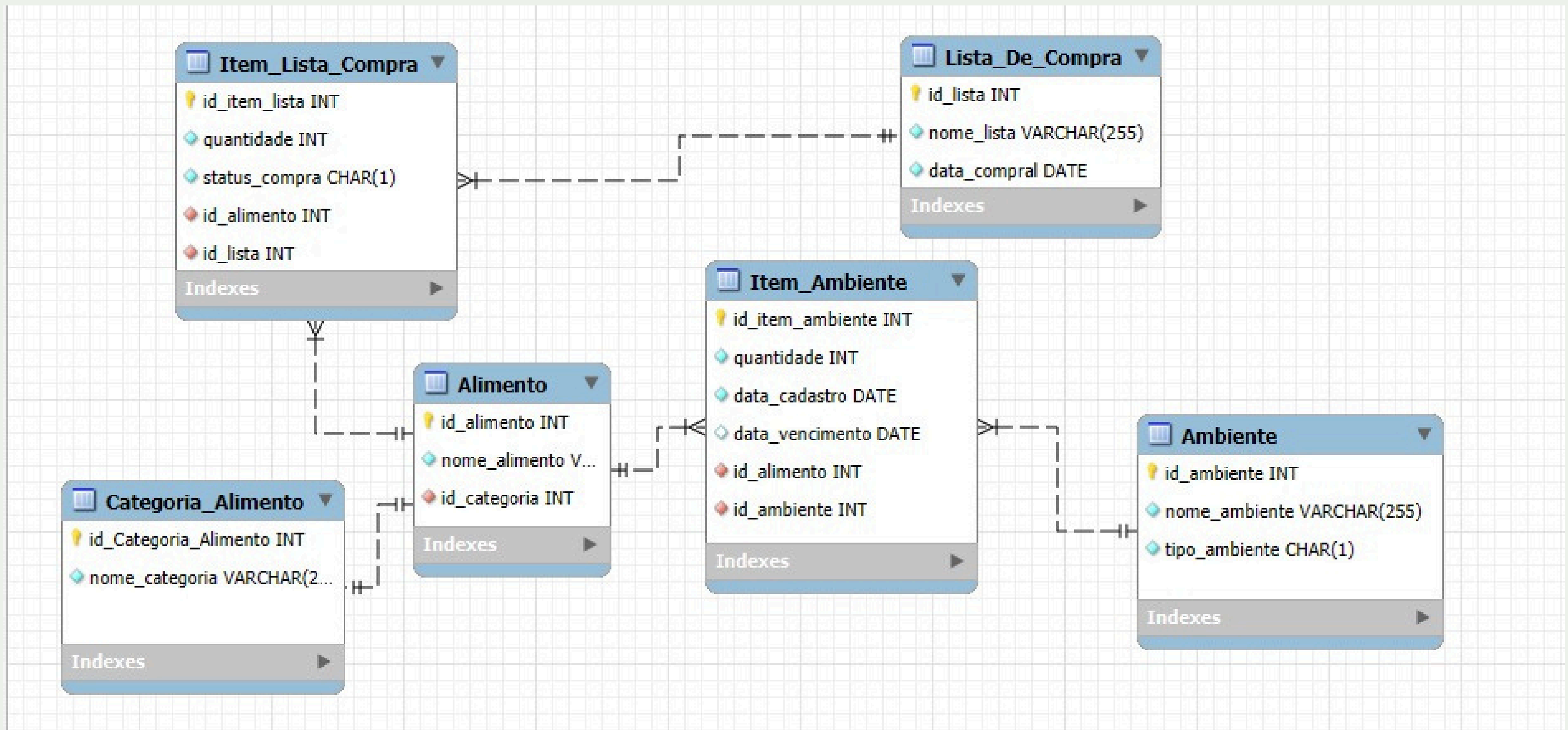
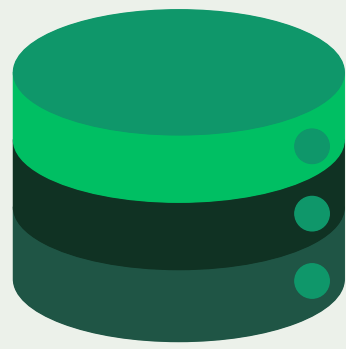
Permitir o acesso

Conectar

Exemplos

Espelhamento no Microsoft Fabric

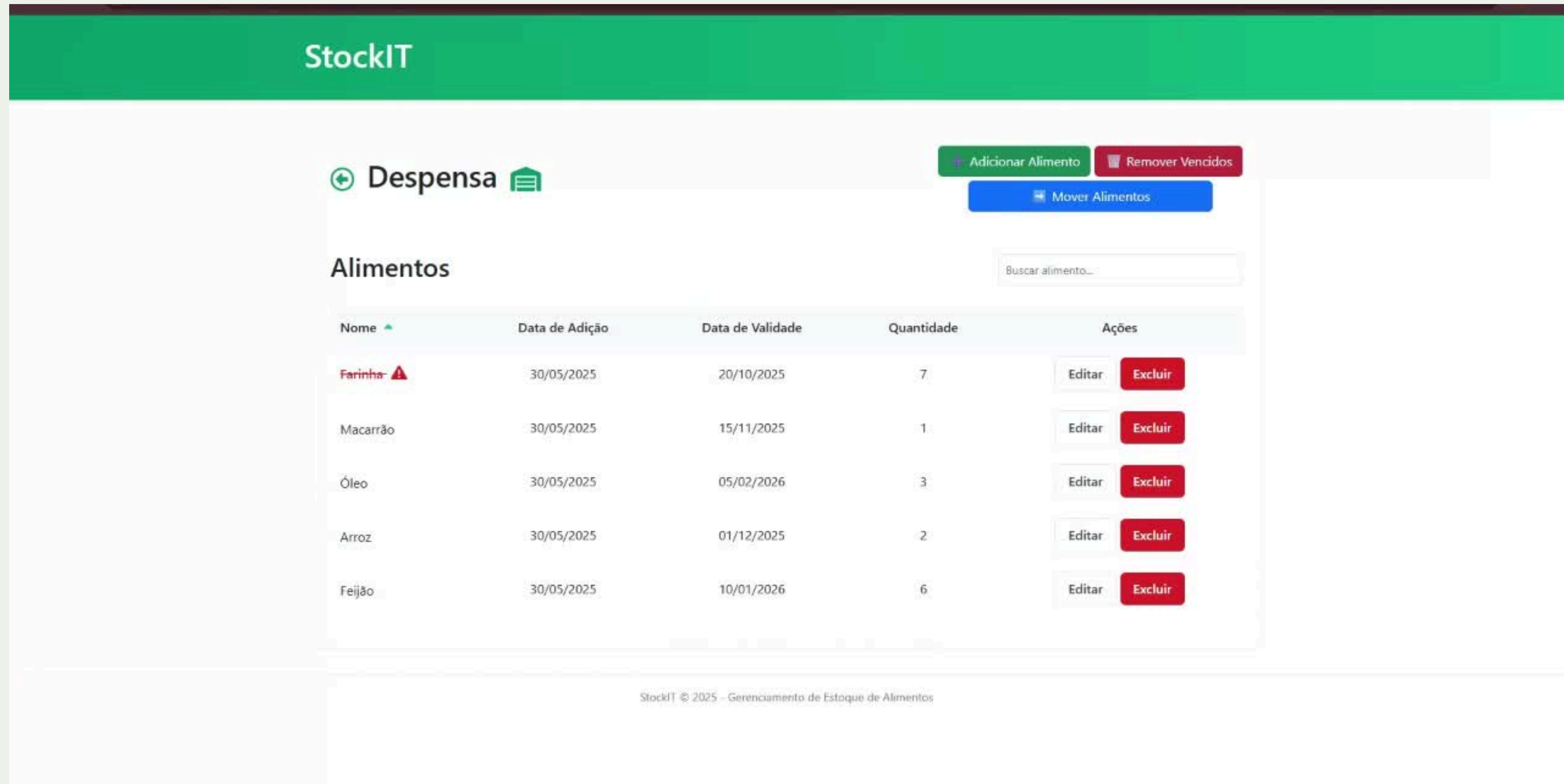
MODELO DE IMPLEMENTAÇÃO



ESTRUTURA ECLIPSE

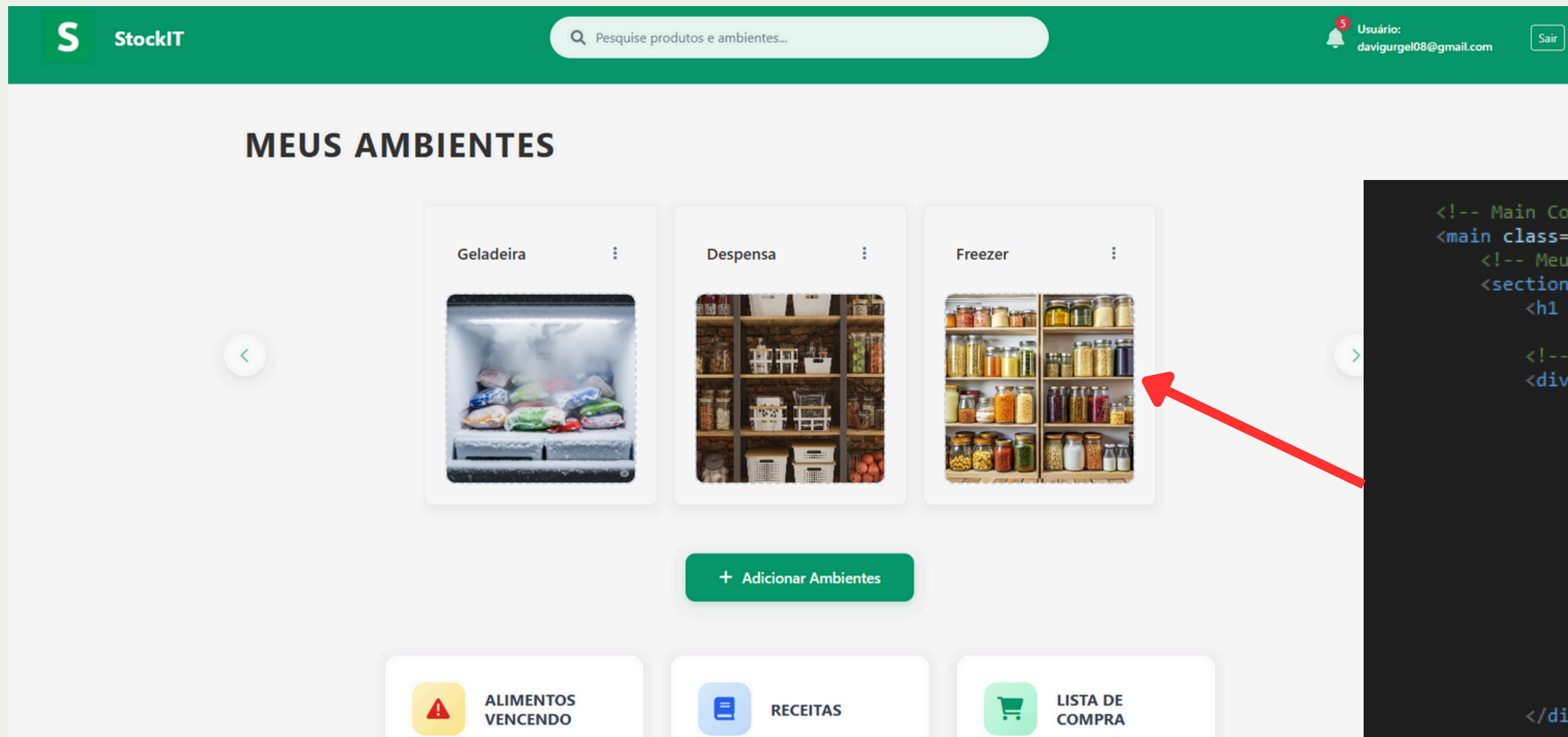
```
▼ [Icon] > plmg-cc-ti2-2025-2-g03-stockit [plmg-cc-ti2-2025-2-g03-stockit master]
  > [Icon] Artefatos
  ▼ [Icon] >Codigo
    ▼ [Icon] > stock-it
      ▼ [Icon] > src
        ▼ [Icon] > main
          ▼ [Icon] java
            ▼ [Icon] app
              [Icon] Principal.java
            ▼ [Icon] dao
              [Icon] AlimentoDAO.java
              [Icon] AmbienteDAO.java
              [Icon] CategoriaAlimentoDAO.java
              [Icon] DAO.java
              [Icon] ItemAmbienteDAO.java
              [Icon] ItemListaCompraDAO.java
              [Icon] ListaDeCompraDAO.java
            ▼ [Icon] model
              [Icon] Alimento.java
              [Icon] Ambiente.java
              [Icon] CategoriaAlimento.java
              [Icon] ItemAmbiente.java
              [Icon] ItemListaCompra.java
              [Icon] ListaDeCompra.java
            ▼ [Icon] service
              [Icon] AlimentoService.java
              [Icon] AmbienteService.java
              [Icon] CategoriaAlimentoService.java
              [Icon] ItemAmbienteService.java
              [Icon] ItemListaCompraService.java
              [Icon] ListaDeCompraService.java
          > [Icon] util
        ▼ [Icon] > resources
          ▼ [Icon] > public
            > [Icon] > assets
            > [Icon] > modulos
          > [Icon] scripts
            [Icon] log4jdbc.log4j2.properties
            [Icon] simplelogger.properties
        > [Icon] target
          [Icon] pom.xml
          [Icon] README.md
```

FLUXO DE APLICAÇÃO



link: <https://youtu.be/jnlaFRjaCOo>

FLUXO, DE APLICAÇÃO MÉTODO GET



```
<!-- Main Content -->
<main class="main-content">
  <!-- Meus Ambientes Section -->
  <section class="ambientes-section">
    <h1 class="section-title">MEUS AMBIENTES</h1>

    <!-- Carousel Container -->
    <div class="carousel-container">
      <button class="carousel-btn carousel-btn-prev" id="carousel-prev">
        <i class="fa-solid fa-chevron-left"></i>
      </button>

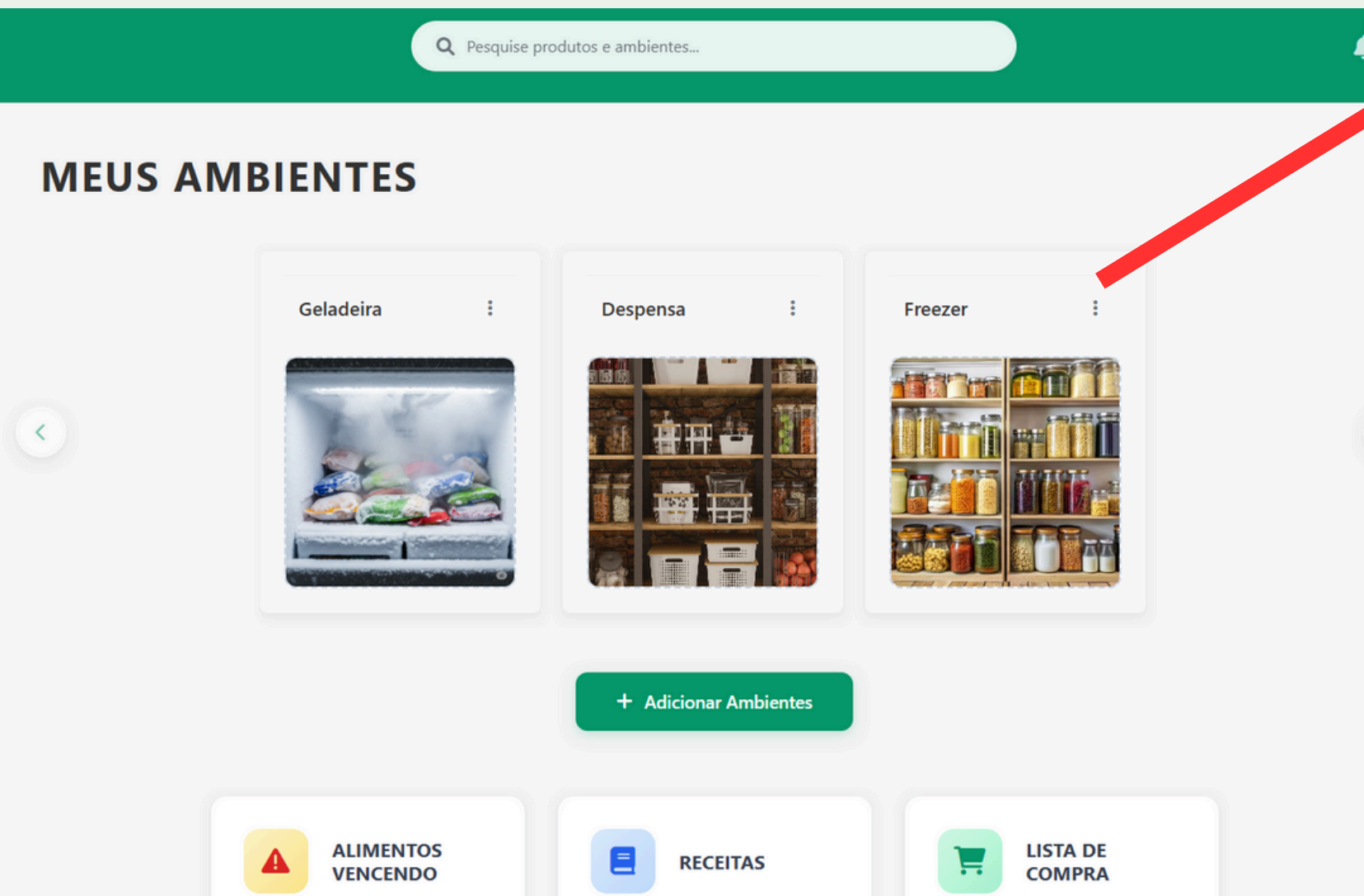
      <div class="carousel-wrapper">
        <div class="ambientes-carousel" id="ambientes-container">
          <!-- Os ambientes serão carregados dinamicamente aqui -->
        </div>
      </div>

      <button class="carousel-btn carousel-btn-next" id="carousel-next">
        <i class="fa-solid fa-chevron-right"></i>
      </button>
    </div>

    <div class="add-ambiente-container">
      <button id="btn-crudAmbiente" class="add-ambiente-btn" data-bs-toggle="modal"
        data-bs-target="#modalCadastroAmbiente">
        <i class="fa-solid fa-plus"></i>
        Adicionar Ambientes
      </button>
    </div>
  </div>
</section>
```


FLUXO, DA APLICAÇÃO MÉTODO GET

Ao selecionar um ambiente,
chama a função no JS



```
// Função para carregar o nome do ambiente
async function carregarAmbiente() {
  try {
    const response = await fetch(`${apiUrl}/ambientes/${ambienteId}`);
    const ambiente = await response.json();
    document.getElementById('nome-ambiente').textContent = ambiente.nome;
    document.title = `StockIT - ${ambiente.nome}`;

    const tipoAmbiente = ambiente.tipo;
    const iconeAmbiente = await obterIconeAmbiente(tipoAmbiente);
    document.getElementById('icone-ambiente').className = iconeAmbiente;
    preencherSelectMover()
  } catch (error) {
    console.error('Erro ao carregar o ambiente:ou JSON SERVER Offline', error);
  }
}

// Função para carregar os alimentos do ambiente
async function carregarAlimentos() {
  try {
    const response = await fetch(`${apiUrl}/ambientes/${ambienteId}`);
    const ambiente = await response.json();
    const corpoTabela = document.getElementById('corpo-tabela-alimentos');
    corpoTabela.innerHTML = '';
    linhasTabela = [];

    ambiente.itens.forEach(async (item, index) => {
      const alimentoResponse = await fetch(`${apiUrl}/alimentos/${item.alimentoId}`);
      const alimento = await alimentoResponse.json();
      const validade = conferirValidade(formatarData(item.vencimento)) == true ? "" : "fa-solid fa-triangle-exclamation";
      const linha = document.createElement('tr');
      linha.innerHTML = `
        <td id="td-nome">${alimento.nome} ${alimento.tipo || ""} <i id = icone-vencimento class ="${validade}"><i> </td>
        <td>${formatarData(item.cadastro)}</td>
        <td>${formatarData(item.vencimento)}</td>
        <td>${item.quantidade} </td>
        <td>
          <button class="botao-secundario" onclick="openModal('modal-editar','form-editar', ${index} , ${ambienteId})">Editar</button>
          <button class="botao-perigo" onclick = "openModal('modal-excluir', '',${index}, ${ambienteId})">Excluir</button>
        </td>
      `;
      if (validade) {
        const nome = linha.querySelector('td');
        nome.classList.add("alimento-vencido");
      }
      corpoTabela.appendChild(linha);
      linhasTabela.push(linha);
    });
  }
}
```

FLUXO, DA APLICAÇÃO MÉTODO GET

Aplicação

```
get("/apistockit/ambientes/:id", (request, response) -> ambienteService.buscar(request, response));
```

Service

```
// READ - por id
public Object buscar(Request req, Response res) {
    res.type("application/json");
    try {
        int id = Integer.parseInt(req.params(":id"));
        Ambiente ambiente = dao.getPorId(id);

        if (ambiente != null) {
            return new Gson().toJson(ambiente);
        } else {
            res.status(404);
            return "{\"error\":\"Ambiente não encontrado\"}";
        }
    } catch (Exception e) {
        res.status(500);
        return "{\"error\":\"" + e.getMessage() + "\"}";
    }
}
```

DAO

```
// READ - por ID
public Ambiente getPorId(int id) {
    Ambiente a = null;
    try {
        String sql = "SELECT * FROM ambiente WHERE id_ambiente = ?";
        PreparedStatement st = conexao.prepareStatement(sql);
        st.setInt(parameterIndex:1, id);
        ResultSet rs = st.executeQuery();
        if (rs.next()) a = mapear(rs);
        rs.close();
        st.close();
    } catch (SQLException e) {
        System.err.println("Erro ao buscar ambiente por id: " + e.getMessage());
    }
    return a;
}
```

FLUXO, DA APLICAÇÃO MÉTODO GET

DAO

Banco de Dados

Query

Query History

1

```
select * from ambiente where id_ambiente = 2
```

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	id_ambiente [PK] integer	nome_ambiente character varying (255)	tipo_ambiente character (1)
1	2	Freezer	F

```
// READ - por ID
public Ambiente getPorId(int id) {
    Ambiente a = null;
    try {
        String sql = "SELECT * FROM ambiente WHERE id_ambiente = ?";
        PreparedStatement st = conexao.prepareStatement(sql);
        st.setInt(parameterIndex:1, id);
        ResultSet rs = st.executeQuery();
        if (rs.next()) a = mapear(rs);
        rs.close();
        st.close();
    } catch (SQLException e) {
        System.err.println("Erro ao buscar ambiente por id: " + e.getMessage());
    }
    return a;
}
```

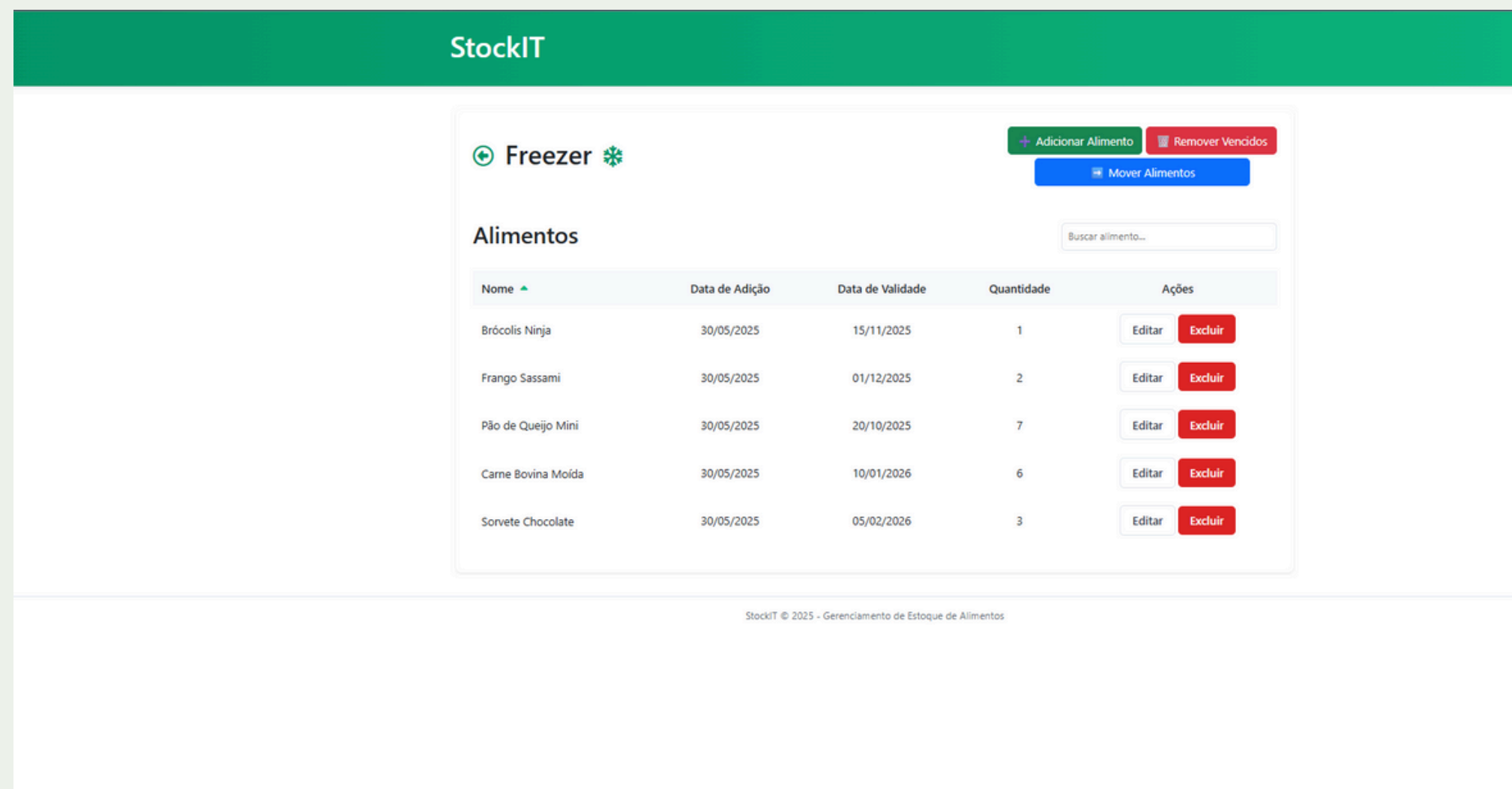
Service

```
// READ - por id
public Object buscar(Request req, Response res) {
    res.type("application/json");
    try {
        int id = Integer.parseInt(req.params(":id"));
        Ambiente ambiente = dao.getPorId(id);

        if (ambiente != null) {
            return new Gson().toJson(ambiente);
        } else {
            res.status(404);
            return "{\"error\":\"Ambiente não encontrado\"}";
        }
    } catch (Exception e) {
        res.status(500);
        return "{\"error\":\"" + e.getMessage() + "\"}";
    }
}
```

FLUXO DA APLICAÇÃO MÉTODO POST

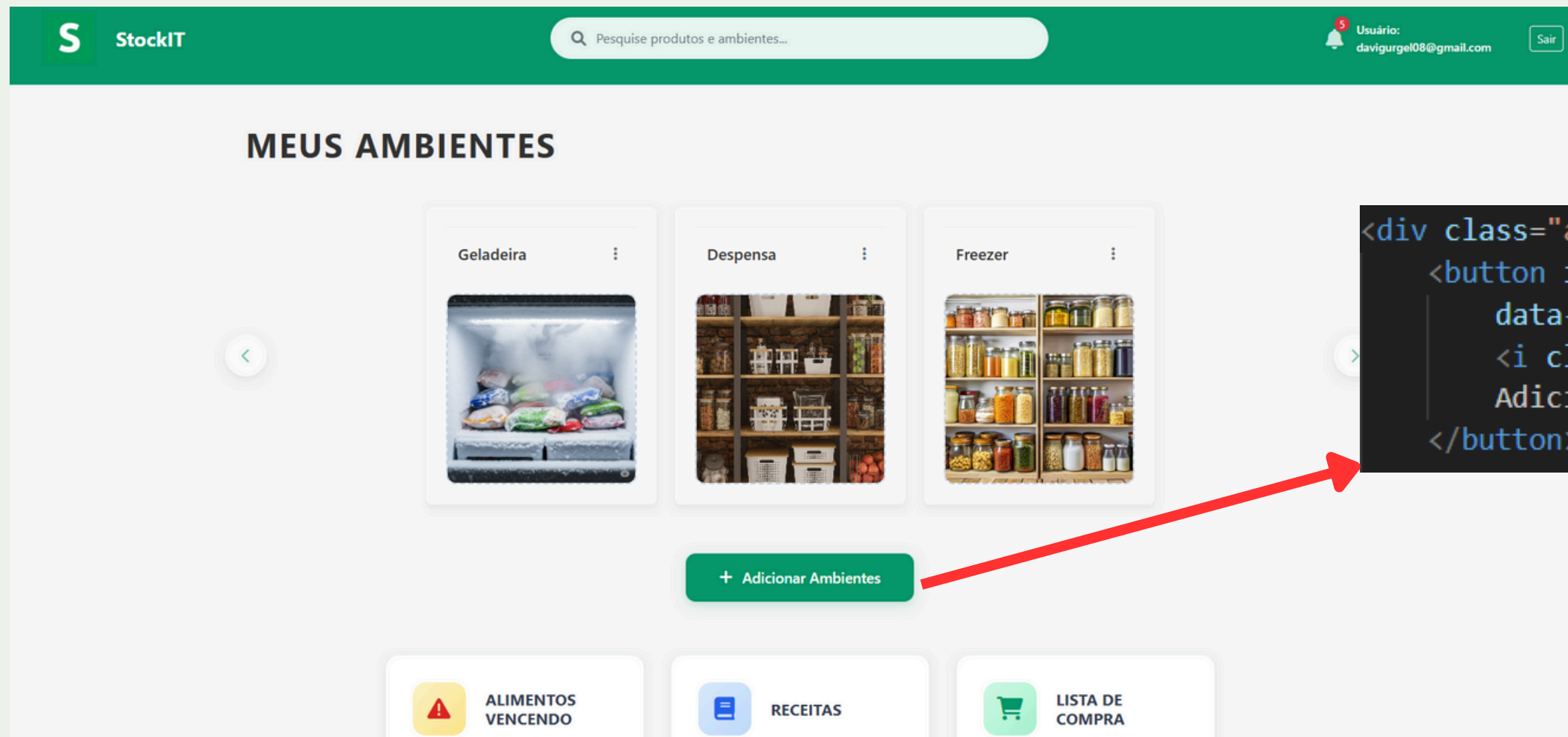
**Neste ponto ao voltar da requisição
nosso retorno é a abertura do
espaço de ambiente selecionado**



```
<body>
  <header class="cabecalho">
    <div class="cabecalho-conteudo">
      <div class="logo-container">
        <a id="nome-projeto" href=" ../home-page/home.html">
          <h1>StockIT</h1>
        </a>
      </div>
    </div>
  </header>

  <main class="conteudo-principal">
    <div class="container-conteudo">
      <div class="container-cabecalho">
        <div class="container-cabecalho-esquerda">
          <a href=" ../home-page/home.html" id="botao-voltar" class="botao-voltar"><i
            class="fa-regular fa-circle-left"></i></a>
          <h2 id="nome-ambiente">Carregando ambiente...</h2>
          <i id="icone-ambiente"></i>
        </div>
        <div class="container-cabecalho-direita">
          <div class="container-cabecalho-esquerda-superior">
            <button id="botao-adicionar-alimento"
              onclick="openModal('modal-cadastro' , 'form-cadastro')" class="btn tn btn-success">
              ➕ Adicionar Alimento</button>
            <button id="botao-remover-vencido" onclick="openModal('modal-excluir-vencidos')" type="button"
              class=" btn btn-danger">
              🗑 Remover Vencidos</button>
          </div>
          <div class="container-cabecalho-esquerda-inferior">
            <button id="botao-mover" onclick="openModal('modal-mover-alimentos')" type="button"
              class=" botao-primario btn btn-primary">
              ➡ Mover Alimentos</button>
          </div>
        </div>
      </div>
    </div>
  </main>
</body>
```

FLUXO DE APLICAÇÃO MÉTODO POST

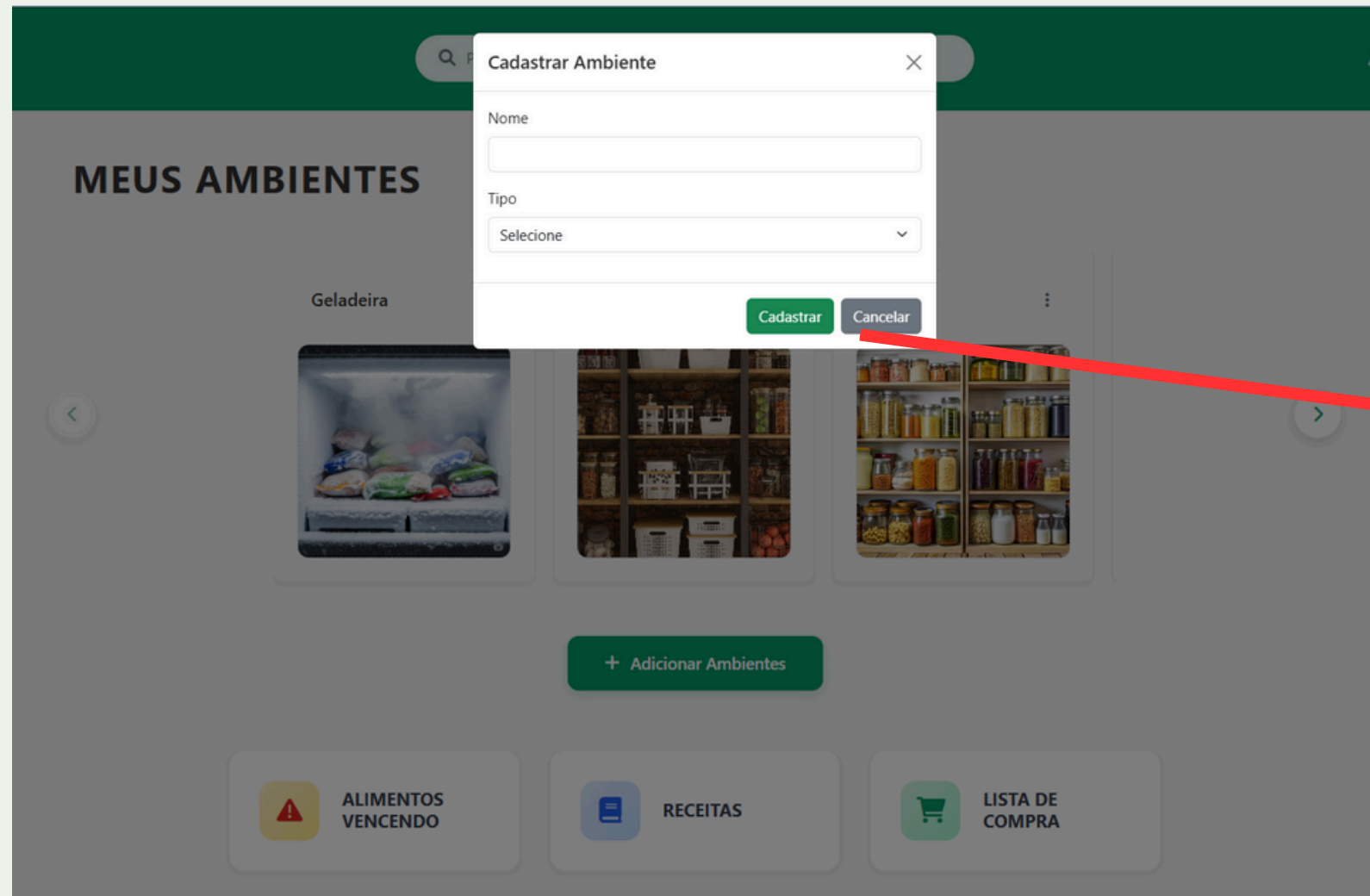


```
<div class="add-ambiente-container">  
  <button id="btn-crudAmbiente" class="add-ambiente-btn" data-bs-toggle="modal"  
    data-bs-target="#modalCadastroAmbiente">  
    <i class="fa-solid fa-plus"></i>  
    Adicionar Ambientes  
  </button>  
</div>
```

Ao clicar em adicionar um ambiente é aberto um modal

FLUXO DE APLICAÇÃO

MÉTODO POST



Ao cadastrar no modal, chama a função no JS

```
//CADASTRAR AMBIENTE
document.getElementById("formCadastrarAmbiente").addEventListener("submit", async (e) => {
  e.preventDefault();

  const nome = document.getElementById("nomeAmbienteModal").value.trim();
  const tipo = document.getElementById("tipoAmbienteModal").value;

  if (!nome || !tipo) {
    Swal.fire({
      icon: "warning",
      title: "Preencha todos os campos obrigatórios",
      confirmButtonColor: "#dc3545"
    });
    return;
  }

  // Mapeamento das imagens pelos tipos
  const imagensPorTipo = {
    tipo0: "tipo0.png",
    tipo1: "tipo1.png",
    tipo2: "tipo2.png",
    tipo3: "tipo3.png"
  };

  // Caminho final da imagem baseada no tipo
  const imagemFinal = `../../assets/images/ambientes-imagens/${imagensPorTipo[tipo]}`;

  const novoAmbiente = {
    nome,
    tipo,
    imagem: imagemFinal,
    itens: []
  };

  try {
    const res = await fetch(`${apiUrl}/ambientes`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(novoAmbiente)
    });

    if (res.ok) {
      Swal.fire({
        icon: "success",
        title: "Ambiente cadastrado com sucesso!",
        confirmButtonColor: "#28a745"
      }).then((result) => {
        if (result.isConfirmed) {
          document.getElementById("formCadastrarAmbiente").reset();
          const modal = bootstrap.Modal.getInstance(document.getElementById("modalCadastroAmbiente"));
          modal.hide();
          location.reload();
        }
      });
    } else {
      throw new Error();
    }
  } catch {
    Swal.fire({
      icon: "error",
      title: "Erro ao cadastrar ambiente",
      text: "Tente novamente.",
      confirmButtonColor: "#dc3545"
    });
  }
});
```

FLUXO DE APLICAÇÃO MÉTODO POST

Aplicação

```
post("/apistockit/ambientes/insert", (request, response) -> ambienteService.insert(request, response));
```

Service

```
public boolean insert(Ambiente a) throws SQLException {  
    validar(a);  
    return dao.inserir(a);  
}
```

DAO

```
public boolean inserir(Ambiente a) {  
    boolean status = false;  
    try {  
        String sql = "INSERT INTO ambiente (nome_ambiente, tipo_ambiente) VALUES (?, ?)";  
        PreparedStatement st = conexao.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);  
        st.setString(1, a.getNomeAmbiente());  
        st.setString(2, String.valueOf(a.getTipoAmbiente()));  
        st.executeUpdate();  
  
        try (ResultSet rs = st.getGeneratedKeys()) {  
            if (rs.next()) a.setId(rs.getInt(1)); // id = id_ambiente  
        }  
  
        st.close();  
        status = true;  
    } catch (SQLException e) {  
        System.err.println("Erro ao inserir ambiente: " + e.getMessage());  
    }  
    return status;  
}
```

FLUXO DE APLICAÇÃO MÉTODO POST

DAO

```
INSERT INTO ambiente (nome_ambiente, tipo_ambiente)
VALUES ('Frigobar', 'F');

SELECT * FROM ambiente
```

```
public boolean inserir(Ambiente a) {
    boolean status = false;
    try {
        String sql = "INSERT INTO ambiente (nome_ambiente, tipo_ambiente) VALUES (?, ?)";
        PreparedStatement st = conexao.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
        st.setString(1, a.getNomeAmbiente());
        st.setString(2, String.valueOf(a.getTipoAmbiente()));
        st.executeUpdate();

        try (ResultSet rs = st.getGeneratedKeys()) {
            if (rs.next()) a.setId(rs.getInt(1)); // id = id_ambiente
        }

        st.close();
        status = true;
    } catch (SQLException e) {
        System.err.println("Erro ao inserir ambiente: " + e.getMessage());
    }
    return status;
}
```

Service

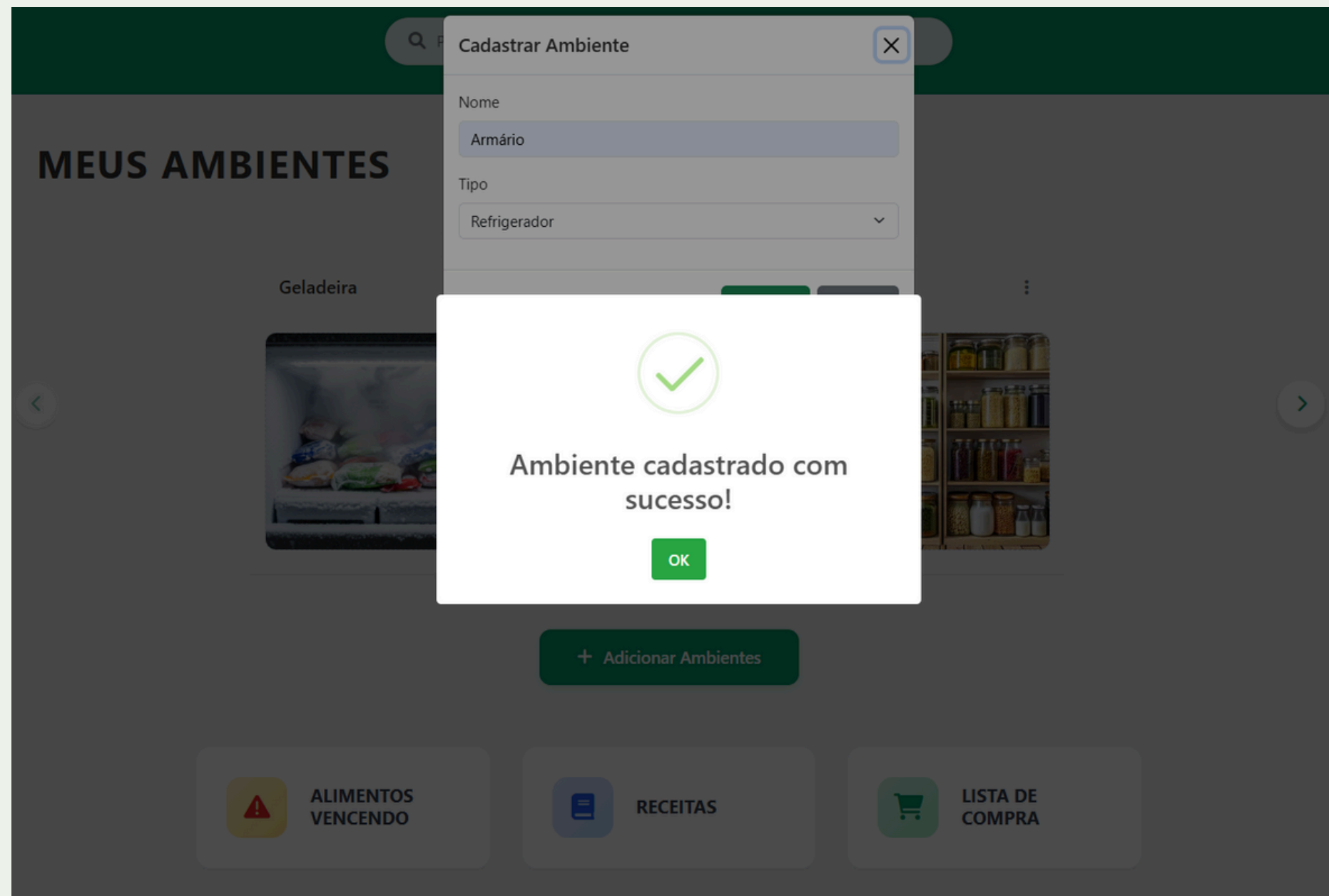
```
public boolean insert(Ambiente a) throws SQLException {
    validar(a);
    return dao.inserir(a);
}
```

Banco de dados

Output Messages Notifications		
SQL		
id_ambiente [PK] integer	nome_ambiente character varying (255)	tipo_ambiente character (1)
1	Geladeira	G
2	Freezer	F
3	Despensa	D
4	Frigobar	F

FLUXO DE APLICAÇÃO MÉTODO POST

Neste ponto ao voltar da requisição nosso retorno é a mensagem de cadastro concluído



```
try {
  const res = await fetch(`${apiUrl}/ambientes`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(novoAmbiente)
  });

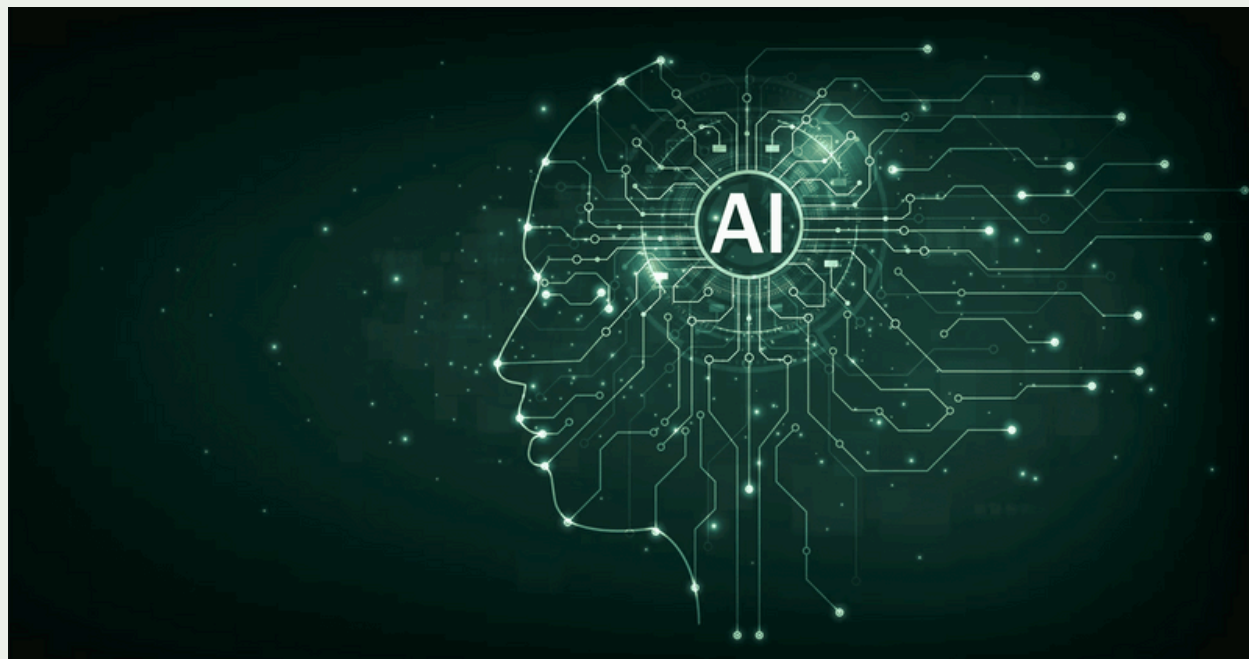
  if (res.ok) {
    Swal.fire({
      icon: "success",
      title: "Ambiente cadastrado com sucesso!",
      confirmButtonColor: "#28a745"
    }).then((result) => {
      if (result.isConfirmed) {
        document.getElementById("formCadastrarAmbiente").reset();
        const modal = bootstrap.Modal.getInstance(document.getElementById("modalCadastroAmbiente"));
        modal.hide();
        location.reload();
      }
    });
  } else {
    throw new Error();
  }
} catch {
  Swal.fire({
    icon: "error",
    title: "Erro ao cadastrar ambiente",
    text: "Tente novamente.",
    confirmButtonColor: "#dc3545"
  });
}
```


SISTEMAS & IA INTELIGENTES

Tipo de sistema: Automação do cadastro de itens no estoque

Inteligência aplicada: Leitura e interpretação automática de cupons fiscais (imagem) com extração de produtos, quantidades.

Proposição de valor: Reduz erros manuais, atualiza o estoque automaticamente, gera eficiência e economia de tempo



Aprendizado: Modelo treinado previamente com base de dados gerados e com novos cadastros

Técnicas utilizadas: OCR (reconhecimento óptico de caracteres) Correspondência de texto e análise semântica Regras automáticas de decisão

Entradas: Cupom fiscal (imagem)

Saídas: JSON com dados estruturados + atualização no banco de estoque

Fornecedor de IA: Microsoft Azure

Serviços utilizados: Cloud Vision (OCR) e Natural Language API

AGRADEÇEMOS
A ATENÇÃO



ESPAÇO PARA DÚVIDAS

